

Generic Behavior Tree

Introduction

Generic Behavior Tree is a Unity asset that provides a flexible and reusable behavior tree implementation. Behavior trees are used to define the decision-making process of AI characters in games. This asset allows you to create complex AI behaviors by combining simple tasks and decisions in a tree structure.

Classes and Methods

Tree Class

The `Tree` class is the base class for all behavior trees. It is responsible for updating the tree and managing the root node.

Methods:

- `Start()`: Initializes the root node by calling the `SetupTree` method.
- `Update()`: Evaluates the root node each frame.
- `SetupTree()`: Abstract method to be overridden by subclasses to define the tree structure.

Node Class

The `Node` class represents a single node in the behavior tree. Nodes can have children and manage their own state.

Properties:

- `NodeState state`: The current state of the node (SUCCESS, FAILURE, RUNNING, IDLE).
- `Node parent`: The parent node.
- `List<Node> children`: The list of child nodes.

Methods:

- `Evaluate()`: Virtual method to evaluate the node's behavior.
- `SetData(string key, object value)`: Stores data in the node's context.
- `GetData(string key)`: Retrieves data from the node's context.

- `ClearData(string key)`: Clears data from the node's context.

Sequence Class

The Sequence class is a composite node that evaluates its children in sequence. If any child fails, the sequence fails. If all children succeed, the sequence succeeds.

Methods:

- `Evaluate()`: Evaluates each child in sequence.

Selector Class

The Selector class is a composite node that evaluates its children until one succeeds. If any child succeeds, the selector succeeds. If all children fail, the selector fails.

Methods:

- `Evaluate()`: Evaluates each child in order.

Node States

In the Generic Behavior Tree, nodes can be in one of the following states:

- **SUCCESS**: Indicates that the node has completed its task successfully. The parent node can then decide to move on to the next task or complete its evaluation.
- **FAILURE**: Indicates that the node has failed to complete its task. The parent node may stop further evaluation or try another task.
- **RUNNING**: Indicates that the node is currently executing its task and has not yet finished. The parent node will continue to evaluate this node in subsequent frames.

State Transitions

Nodes transition between states based on their evaluation logic. Here are some examples of how state transitions work within different node types:

Sequence Node:

- If a child node returns FAILURE, the sequence node stops evaluation and returns FAILURE.
- If a child node returns SUCCESS, the sequence node continues to the next child.

- If a child node returns RUNNING, the sequence node stops evaluation and returns RUNNING.
- If all child nodes return SUCCESS, the sequence node returns SUCCESS.

Selector Node:

- If a child node returns SUCCESS, the selector node stops evaluation and returns SUCCESS.
- If a child node returns FAILURE, the selector node continues to the next child.
- If a child node returns RUNNING, the selector node stops evaluation and returns RUNNING.
- If all child nodes return FAILURE, the selector node returns FAILURE.

Example Usage of Node States: **Check /Source/Test/Nodes** for more about States.

Usage

To use the Generic Behavior Tree in your Unity project, follow these steps:

1. Create a new script that inherits from `Tree` and override the `SetupTree` method to define the behavior tree structure.

```
public class AIBehavior : Tree
{
    protected override Node SetupTree()
    {
        // Code...
    }
}
```

2. Add the custom script to a `GameObject` in your scene.
3. Implement the necessary `Node` classes for your AI behavior.

4. For example, lets create Node for NPC to patrolling (Walking around the navmesh)

```
using GenericBehaviourTree;
using UnityEngine;
using UnityEngine.AI;

public class PatrolBehavior : Node
{
    public PatrolBehavior()
    {
    }
    public override NodeState Evaluate()
    {
        // Your Moving Logic
        return NodeState.RUNNING;
    }
}
```

5. You can write whatever logic you want to let agent walks and using any sort of system you will find our demo Patrol included in the asset.
6. Now let's add it to our `AIBehaviour` class

```
public class AIBehavior : Tree
{
    protected override Node SetupTree()
    {
        Node root = new Selector(new List<Node>
        {
            new PatrolBehavior(...)
        });
        return root;
    }
}
```

7. Now after saving and running this simple tree the NPC will start his sequence.

8. If you have action that require more than one Node you will need to use the Sequence and create a List of Nodes for example, as provided in the asset demo the NPC will Looking for Enemy then if he found in his area will walk to it and then after he will attempt to attack the target everything just as you want to be and how you want to apply your logic.

```
public class AIBehavior : Tree
{
    protected override Node SetupTree()
    {
        Node root = new Selector(new List<Node>
        {
            new Sequence(new List<Node>() {
                new LookingForEnemy(transform),
                new WalkToEnemy(transform),
                new AttackEnemy(transform)
            }),
            new PatrolBehavior(transform)
        });
        return root;
    }
}
```

This example demonstrates how to create a custom behavior tree for an AI character. The AI Behavior class inherits from Tree and defines a selector node with a sequence of tasks to look for an enemy, walk to the enemy, and attack the enemy. If none of these tasks succeed, the AI will patrol the area.

FAQs

Q: How do I add custom nodes?

A: To add custom nodes, create a new class that inherits from Node and override the Evaluate method to define the node's behavior.

Q: Can I use this asset with existing Unity AI components?

A: Yes, you can integrate this asset with any Unity's assets.